Guide to Geospatial Analysis

Part 1: What is Geospatial Analysis?

The Concept

Imagine you have a list of sales made by a store. That is **Data**. Now, imagine you put those sales on a map to see *where* they happened. That is **Geospatial Data**.

Geospatial Analysis is simply the science of answering questions using location. Instead of just asking "How many landslides happened?" we ask:

- **Where** did they happen?
- **Why** there? (Maybe because of mountains or rain?)
- **What** is near them? (Schools? Hospitals?)

The Toolbox (Python Libraries)

In your notebook, we use specific tools (libraries) to do this work. Think of them like a construction crew:

- **Pandas:** The Accountant. Great at tables and math, but doesn't know what a map is.
- **GeoPandas:** The Accountant's Cool Cousin. Can handle tables *and* understands maps/shapes.
- **Shapely:** The Architect. Draws the shapes (Points, Lines, Polygons).
- **Folium:** The Web Designer. Makes interactive maps you can zoom and click on.
- **Matplotlib:** The Artist. Draws static pictures (graphs and maps).

Part 2: Understanding Geographic Data

The Concept: Coordinates

To find anything on Earth, we use a grid system:

- **Latitude:** How far North or South you are (like climbing a ladder).
  - *Example:* 19.08°N (Mumbai).
- **Longitude:** How far East or West you are.
  - *Example:* 72.88°E (Mumbai).

The Shapes

- **Point (0D):** A single dot. (e.g., A house, a person, a landslide).
- **Line (1D):** Connecting dots. (e.g., A road, a river).
- **Polygon (2D):** An enclosed shape. (e.g., The border of India, a lake).

The Code: Creating a GeoDataFrame

A `GeoDataFrame` is just a table (like Excel) that has a special column called `geometry`.

```
 # We take a regular table (df) with 'latitude' and 'longitude' columns
# We zip them together to make pairs: (lon, lat)
# Note: Computers usually prefer X (Longitude) before Y (Latitude)!
geometry = [Point(lon, lat) for lon, lat in zip(df['longitude'], df['latitude'])]

# Now we convert the regular table to a GeoDataFrame
gdf = gpd.GeoDataFrame(df, geometry=geometry, crs='EPSG:4326')
```

Translation: "Take these number pairs, turn them into 'Point' shapes, and tell the computer these are GPS coordinates."

Part 3: Coordinate Reference Systems (CRS)

The Concept: The "Address System"

The Earth is round (3D). Maps are flat (2D). A **CRS** is the set of rules that tells the computer how to translate the round Earth onto a flat screen.

- **Geographic CRS (EPSG:4326 - WGS84):** Uses **Degrees**. This is what GPS uses. It measures angles from the center of the Earth.
- **Projected CRS (EPSG:3857 - Web Mercator):** Uses **Meters**. This flattens the world so we can measure distances in meters/feet. Used by Google Maps.

Crucial Rule: If you want to measure distance in meters, you **must** convert your data to a Projected CRS first.

The Code: Transforming CRS

```
# Check the current system (It says WGS84 / Degrees)
print(gdf.crs)

# Convert to Web Mercator (Meters)
# The computer does complex math to flatten the curved coordinates
gdf_mercator = gdf.to_crs('EPSG:3857')
```

Part 4: Map Projections (The "Orange Peel" Problem)

The Concept

Imagine peeling an orange in one piece and trying to flatten the peel on a table. You can't do it without tearing it or stretching it. Maps are the same. You cannot make a flat map of a round Earth without distorting *something*.

- **Mercator:** Preserves **Shapes** (Countries look the right shape), but distorts **Size** (Greenland looks huge, but it's actually small).
- **Equal-Area:** Preserves **Size** (Countries are the right size), but distorts **Shape** (They look squished).

The Code: Visualizing Distortions

The notebook plots the exact same data using different mathematical rules (projections).

```
# We change the CRS to 'Robinson' or 'Mollweide' and plot it
sample.to_crs('ESRI:54009').plot()
```

Result: You see the map stretch and squash differently depending on the math used.

Part 5: Data Formats (Vector vs. Raster)

Vector Data

Think "Connect the Dots."

- Uses coordinates to draw smooth points, lines, and shapes.
- **Examples:** Shapefiles (`.shp`), GeoJSON.
- **Best for:** Borders, roads, landslide locations.

Raster Data

Think "Digital Photograph" or "Minecraft."

- A grid of colored squares (pixels).
- **Examples:** TIFF, JPEG, Satellite imagery.
- **Best for:** Temperature maps, elevation, satellite photos.

Part 6: Spatial Joins & Queries

The Concept

This is the magic of geospatial analysis. We don't join data by ID numbers; we join them by **location**.

- **Spatial Query:** "Find all points inside this box."
- **Spatial Join:** "Take these landslide points, check which country polygon they are inside, and add the country name to the point's data."

The Code: Spatial Join

```
# regions_gdf contains polygons (shapes) of continents
# gdf contains points of landslides

# The computer checks every point to see if it is 'within' a polygon
joined = gpd.sjoin(gdf, regions_gdf, how='left', predicate='within')
```

Translation: "Look at every landslide. If it's inside the 'Asia' box, label it 'Asia'."

Part 7: Distance and Buffers

The Problem: The Curvature

You can't use a normal ruler on a globe. The straightest line between India and the USA looks curved on a map.

- **Haversine Formula:** A math formula used to calculate "As the crow flies" distance on a sphere.

The Concept: Buffers

A buffer is a "zone of influence."

- **Example:** "Show me everything within 5km of this hospital."
- **Result:** The point becomes a circle. A line becomes a thick road.

The Code: Creating Buffers

IMPORTANT: You cannot create a buffer in *Degrees*. You must convert to *Meters* (Projected CRS) first.

```
 # 1. Convert to UTM (a system that uses meters)
gdf_utm = gdf.to_crs('EPSG:32643')

# 2. Create a 100km (100,000 meters) circle around Delhi
delhi_buffer = delhi_utm.buffer(100000)

# 3. Find landslides inside that circle
nearby = gdf_utm[gdf_utm.geometry.within(delhi_buffer)]
```

Part 8: Choropleth Mapping

The Concept

A "Choropleth" is just a fancy word for a map where regions are colored based on a number.

- **Example:** A map of the world where countries with more landslides are darker red.

The Code

1. **Aggregate:** Count the landslides per country.
2. **Merge:** Attach those counts to a map of world borders.
3. **Plot:**

```
world_data.plot(
    column='landslide_count',  # The data to decide the color
    cmap='YlOrRd',             # Color scheme (Yellow to Red)
    legend=True                # Show the scale bar
)
```

Part 9: Visualization Techniques

Points

- **Simple:** All dots are red.
- **Categorical:** Landslides are red, floods are blue.
- **Graduated Size:** Big landslides are big dots, small landslides are small dots.

Heatmaps (Hexbins)

If you have too many dots (thousands), they overlap and look like a mess. A heatmap divides the map into a honeycomb grid and counts how many dots are in each cell.

```
# Hexbin plot
axes[1,1].hexbin(x, y, gridsize=20)
```

Part 10: Geocoding

The Concept

Computers don't know where "Taj Mahal" is. They only know `27.17°N, 78.04°E`.

- **Geocoding:** Converting an Address ("Taj Mahal") $\rightarrow$ Coordinates (27.17, 78.04).
- **Reverse Geocoding:** Converting Coordinates $\rightarrow$ Address.

The Code (Geopy)

```
from geopy.geocoders import Nominatim
geolocator = Nominatim(user_agent="my_app")

# Address to numbers
location = geolocator.geocode("Mumbai, India")
print(location.latitude, location.longitude)
```

Part 11: Interactive Maps (Folium)

The Concept

Static maps (images) are boring. We want maps like Google Maps where we can zoom, pan, and click markers to see popups. **Folium** does this.

The Code

```
import folium

# 1. Create the blank map centered on India
m = folium.Map(location=[20.59, 78.96], zoom_start=5)

# 2. Add a marker for Delhi
folium.Marker(
    [28.61, 77.21],
    popup='Delhi'
).add_to(m)

# 3. Show map
m
```

Part 12: Data Integration

The Concept

Geospatial analysis is most powerful when you combine it with *non-spatial* data (Census, Economics, Weather).

- **Example:** You have a map of Countries. You have an Excel sheet of GDP. You combine them to see if rich countries have fewer landslide deaths.

The Code

We use the `.merge()` function, just like in SQL or Excel VLOOKUP.

```
 # Merge the landslide stats with the GDP data
# 'on="country"' means "Match rows where the Country Name is the same"
integrated_data = landslide_stats.merge(gdp_data, on='country', how='inner')

# Now we can plot GDP vs. Landslides
plt.scatter(integrated_data['gdp'], integrated_data['landslides'])
```